INF264

Introduction to Machine Learning

Project 1 – Decision Trees and Random Forests

Oksana Monakhova

okmon0603@uib.no

21.09.2025

Universitetet i Bergen
Fakultet for naturvitenskap og teknologi

Table of Contents

Introduction 3
- **Data Analysis** 3
- **Data Preprocessing** 6

Implementations 6
- **Decision Trees** 6
  - **Notes on Design Choices** 7
- **Random Forest** 8

Model Selection and Evaluation 8
- **Model Selection Process** 8
- **Model Selection Results (Letter Recognition Dataset)** 10
  - **Decision Trees** 10
  - **Random Forest** 10
  - **Comparing Decision Trees and Random Forests** 11
  - **Comparison to Existing Implementations** 11
  - **Feature Importance** 12

Conclusion 13

Introduction

The goal of this project is to explore elementary machine learning algorithms by implementing them from scratch and applying them to real datasets. By doing so, we learn and gain a deeper understanding of how such models work and how design choices can affect their performance. This project focuses especially on decision trees and random forests, which are very commonly used and applied in research, academia, and industry. Furthermore, we will also perform model selection to find the optimal hyperparameters, and these models will be compared to existing algorithms from the *'sklearn' library*.

First, we need to get a better understanding to be able to build this algorithm. A Decision Tree is a supervised learning algorithm that is used for classification and regression. The algorithm works by splitting a dataset into smaller subsets based on features, which are represented as nodes and branches. The goal is to predict a target variable using decision rules.

A Random Forest, on the other hand, is an ensemble method that combines multiple Decision Trees. It builds 'trees' on random subsets of data and aggregates their predictions, and by doing so, it reduces overfitting and overall improves stability and accuracy.

Data Analysis

In this project, we have a given dataset named "*letters.csv*". The dataset consists of 17 columns (16 features + 1 label) and 2000 rows, or 16 numerical features and 2000 examples. The last column contains an integer between 0 and 5 denoting the class label, and the 6 classes correspond to letters from A-F. This dataset is a subsample of the well-known Letter Recognition dataset. Below is a helpful figure to visualize the CSV file.

	x-box	y-box	width	high	onpix	x-bar	y-bar	x2bar	y2bar	xybar	x2ybr	xy2br	x-ege	xegvy	y-ege	yegvx	label
0	6.0	11.0	8.0	8.0	8.0	10.0	7.0	4.0	6.0	9.0	3.0	6.0	3.0	8.0	3.0	8.0	3.0
1	4.0	6.0	6.0	4.0	5.0	7.0	6.0	4.0	4.0	8.0	7.0	11.0	5.0	9.0	3.0	8.0	2.0
2	5.0	8.0	7.0	6.0	4.0	7.0	9.0	3.0	6.0	13.0	6.0	5.0	2.0	9.0	3.0	7.0	5.0
3	4.0	5.0	5.0	7.0	2.0	1.0	14.0	5.0	4.0	12.0	10.0	5.0	0.0	8.0	2.0	5.0	5.0
4	6.0	10.0	6.0	8.0	5.0	6.0	5.0	9.0	7.0	6.0	7.0	6.0	2.0	8.0	10.0	10.0	1.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1995	3.0	6.0	5.0	4.0	6.0	8.0	6.0	5.0	4.0	7.0	7.0	8.0	7.0	9.0	7.0	8.0	1.0
1996	2.0	4.0	3.0	3.0	2.0	10.0	2.0	2.0	2.0	9.0	2.0	9.0	1.0	6.0	2.0	8.0	0.0
1997	2.0	3.0	3.0	2.0	2.0	7.0	7.0	6.0	6.0	7.0	6.0	6.0	5.0	8.0	2.0	7.0	3.0
1998	5.0	9.0	7.0	6.0	9.0	8.0	6.0	5.0	4.0	7.0	7.0	8.0	7.0	9.0	7.0	9.0	1.0
1999	6.0	10.0	8.0	7.0	6.0	10.0	7.0	3.0	7.0	11.0	3.0	6.0	2.0	8.0	5.0	11.0	1.0

Figure 1: A visualization of "letters.csv"

Each row represents one letter, described by a set of numerical features extracted from its image:

- **x-box**: x-coordinate for the upper left corner of the bounding box.
- **y-box**: y-coordinate for the upper left corner of the bounding box.
- **width**: width of the bounding box.
- **high**: height of the bounding box.
- **onpix**: number of "on" (black) pixels inside the bounding box.
- **x-bar**: mean x-position of the pixels.
- **y-bar**: mean y-position of the pixels.
- **x2bar**: variance in x-direction.
- **y2bar**: variance in y-direction.
- **xybar**: covariance between x and y.
- **x2ybr**: third-order moment (x^2y).
- **xy2br**: third-order moment (xy^2).
- **x-ege**: number of edge transitions along the x-direction.
- **xegvy**: variation of x edge transitions across y.
- **y-ege**: number of edge transitions along the y-direction.
- **yegvx**: variation of y edge transitions across x.
- **label**: integer label for the class (0 = A, 1 = B, 2 = C, 3 = D, 4 = E, 5 = F).

This dataset is balanced, and the feature set captures the shapes, sizes, and distributions of pixels in each letter, which enables classification into the letters A-F. See the distribution below in Figure 2.

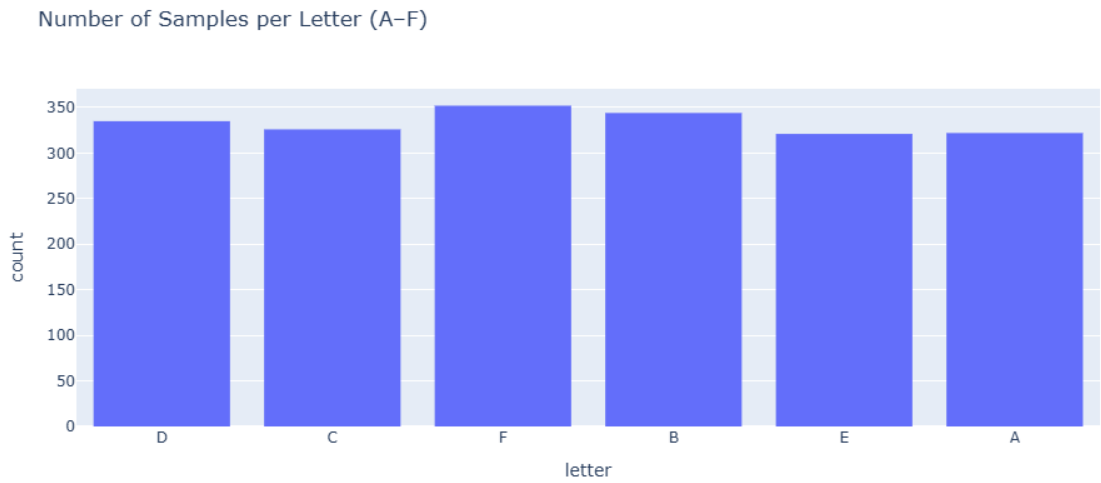

Figure 2: A visualization of number of samples per letter (A-F)

By using the '.describe()' function, we get an overview of the data in the CSV file, see Figure 3.

```
             x-box        y-box        width         high        onpix  \
count  2000.000000  2000.000000  2000.000000  2000.000000  2000.000000
mean      3.819000     7.020000     4.934000     5.196500     3.513500
std       1.698729     3.304097     1.708538     2.143172     2.013665
min       0.000000     0.000000     1.000000     0.000000     0.000000
25%       3.000000     5.000000     4.000000     4.000000     2.000000
50%       4.000000     7.000000     5.000000     5.000000     3.000000
75%       5.000000     9.000000     6.000000     7.000000     5.000000
max      11.000000    15.000000    11.000000    10.000000    12.000000

             x-bar        y-bar        x2bar        y2bar        xybar  \
count  2000.000000  2000.000000  2000.000000  2000.000000  2000.000000
mean      6.721500     7.225000     4.611000     5.637500     8.774000
std       2.269254     2.418533     2.110428     2.355604     2.199169
min       0.000000     0.000000     0.000000     0.000000     4.000000
25%       6.000000     6.000000     3.000000     4.000000     7.000000
50%       7.000000     7.000000     4.000000     6.000000     8.000000
75%       8.000000     8.000000     6.000000     7.000000    11.000000
max      13.000000    15.000000    11.000000    12.000000    14.000000

             x2ybr        xy2br        x-ege        xegvy        y-ege  \
count  2000.000000  2000.000000  2000.000000  2000.000000  2000.000000
mean      5.804500     8.058000     2.528000     8.084000     4.457500
std       2.331523     2.792048     1.696955     1.329973     2.159288
min       0.000000     2.000000     0.000000     2.000000     0.000000
...
min       2.000000     0.000000
25%       7.000000     1.000000
50%       8.000000     3.000000
75%       9.000000     4.000000
max      13.000000     5.000000
```

Figure 3: Overview over describing attributes for "letters.csv"

Data Preprocessing

The dataset is already well-structured and quite numerical, which resulted in having to do no preprocessing. The only processing was loading the dataset from CSV into a DataFrame, then separating the features and labels and splitting into training, validation, and test sets.

Implementations

This project requires implementing both Decision Tree and Random Forest from scratch without using any built-in classifiers.

Decision Trees

I designed a custom yet simple Decision Tree Classifier that consists of multiple helper functions, classes, and components working together. The main methods that were used were responsible for splitting data, calculating impurity with two unique methods, and growing trees recursively depending on optimal and best splits, which is central in the decision tree algorithm.

Nodes were crucial to building this algorithm from scratch since each node contained information about which feature and threshold value to split on and simultaneously directed it to either a left or a right child node. The entire structure is created using the class called 'Node'. This is all located in the _init_ method, which defines and initializes the parameters of the decision tree. These parameters such as, max_depth, min_samples_split, and criterion, are all essential when it comes to controlling how the tree is being built.

Evaluating the quality of a split was done with two impurity measures: "gini" and "entropy". These were implemented in helper functions "gini_index" and "entropy". Entropy measures disorder or uncertainty in a dataset. Entropy values range from 0 to 1, where 0 indicates a perfectly pure node (often referred as a leaf node), and 1 represents maximum disorder. The formula for entropy is as follows:

$$Entropy: H(x) = -\sum p(x) \log_2(p(x)) \tag{1}$$

Similarly, the gini index measures the probability that a randomly chosen sample would be incorrectly classified if labeled according to the class distribution at the node. The values for gini range from 0 to 0.5, where 0 represents a pure node, and 0.5 represents a node with a 50/50 class split. The formula for gini is as follows:

$$Gini: G(p) = 1 - (p_1^2 + p_2^2)^2 \quad (2)$$

Implementing two methods was done because the task asked us to do so, and the method chosen in the code is decided by the "criterion". I have written the code in a way where the user can choose the preferred method by simply changing the criterion to either "gini" or "entropy". As previously said, the chosen method influences how the splits are selected, and therefore, they give different accuracies. Afterwards, the 'information_gain' function calculates the information gain, the order before and after the split, and therefore how much a split improves the model.

The said 'split' method divides the given data based on a given feature and threshold. It results in creating two subsets of data, where one set has feature values that are less than the threshold and one where they are greater than or equal to the threshold.

Additionally, we have the 'best_split' method, which loops through all the possible feature-value combinations to find the split that maximizes the information gain the most, and it does so by evaluating each potential split. The max_feature parameter selects a subset of features to consider when determining the best split, which is important in Random Forest to introduce randomness.

One of the most important parts of the decision tree algorithm is the 'build_three' method. This method is essentially what builds the tree recursively by choosing the best split at each step, resulting in creating left and right child nodes. The tree will continue to build until it reaches a stopping criterion, either in depth or samples, or there's nothing left to split, also known as a leaf node.

After the tree is finished being built, we can start the 'fit' method, which essentially ties everything together and initializes the tree-building process. This method trains the model and accepts different hyperparameters, which will be discussed later. Hyperparameters allow for flexibility when it comes to the depth of the tree and determining which features to use for splits.

When everything is done and finished, we can start the 'predict' method. What happens in this method is that it traverses the tree for each input sample. It does so by following decision rules from the root node to a leaf node. To help, we have the 'predict_input' function, which handles the traversal for individual input and returns the class label at the leaf node.

Notes on Design Choices

Both 'gini' and 'entropy' were used as options for calculating impurity in this code, and this allows for flexibility but also comparison in how the model evaluates the quality of the splits. It is essential to learn and understand how different methods impact the results when used on different datasets and models.

As previously stated, the recursive split is essential, and each node is built based on the best split found. This process will then repeat itself until the stopping criteria are met or it has reached a leaf node. The

stopping criteria are essential to prevent overfitting by limiting the growth of the tree. We have, for instance, 'max_depth' and 'min_samples_split', which help with exactly that and are used as restrictions and limits for the growing tree. They are critical to ensure that the tree grows in the best way possible, avoiding growing too deep or creating splits with too few samples.

We also have the 'max_features' parameter, which selects at random a subset of features used in consideration when splitting a node, which is useful when we are implementing the random forest algorithm. The parameter can have several values: None, sqrt or log2, which decides how the total number of features will be presented. For example, if the value is 'sqrt', then it considers the square root of the total number of features.

Random Forest

Similar to the Decision Tree, the code starts off with the _init_ method, which is what initializes the hyperparameters and sets up the entire structure. The Random Forest has more hyperparameters since it involves training multiple Decision Trees, which requires additional configurations to manage the diversity and behaviour of the trees.

On Random Forest, we want to train our model, but to do so, we need to implement bootstrap. Bootstrapping the training dataset means training different decision trees on subsets of the training set, and due to the replacements, each subset differs from the others. Afterwards, it takes the average of the decision trees, which results in a more stable model that is less likely to overfit.

To fit and train our dataset and labels, we have a function called 'fit(X,y)'. This function uses the samples, which are based on the dimensions of the dataset, to train our model. Afterwards, it iterates over the range of decision trees, which is given by n-estimators, to train each tree individually. After every sample, each tree makes a prediction that is added to a list. Then we use majority voting on the said list for our final prediction.

Model Selection and Evaluation

Model Selection Process

Firstly, we need to import the dataset 'letter.csv', which has been provided to us in the assignment. Then we need to define the features and targets as X and y, respectively, of the given dataset. From scikit-learn, we use the function 'train_test_split' to divide the dataset into four: one for training and testing for both X and y. The data will be used to train and test our model. It is important to note that we fix the seed to 0 and use np.random.seed(seed) so that our results will be reproducible.

Next, we use several methods for our model selection. They go as follows: *K-Fold cross-validation*, *F1-score* and *hyperparameters*.

K-Fold cross-validation is a well-known technique used to estimate how well a model generalizes new data. The technique starts off with splitting the data into *k* parts, which are called folds, hence the name. The split parts will be training data, whilst one part will be the validation data. After the parts have been split, the training fold will be used as a validation fold once. The remaining folds will be used as the training set. This means that all folds will eventually be used as validation once, and the training folds will be used k-1 times.

F1-score will give a score to each iteration with a value between 0 and 1, where 0 is the worst and 1 is the best. This technique is good when evaluating both precision and recall, and the balance between them. To further explain, precision is how many actual correct predictions are divided by all predictions the model deemed correct (i.e., including false positives).

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \tag{3}$$

Recall is the correct predictions divided by all the actual correct answers (i.e., including those the model incorrectly predicted).

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives} \tag{4}$$

F1-score will then be the harmonic mean of the precision and recall.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{5}$$

Therefore, if both the precision and recall are high, then F1 will be high. If one of them is low, then F1 will tremendously fall. To summarize, F1 will punish the imbalance between precision and recall.

We then use F1 to score each iteration of our k-fold cross-validation, and we take the average to get our value, which is done for all combinations of our parameters.

Hyperparameters are defined by the user before running the code. For the custom decision tree, the parameters used were depth, features, and criterion. The Random Forest model uses these parameters, as well as n-estimators, as hyperparameters. There is then a list created of all combinations of hyperparameters, and each combination will be tested and then assigned a value before it's added to a list. The numpy function 'argmax' is used to find the hyperparameters that give the highest F1-score.

To be more precise, when we tune for Decision Tree and Random Forest, we look at max_depth, max_features, and criterion. In max_depth, we look at the values: *None, 5, 10, 15, 20,* and *30*. In max_features, we look at: *'sqrt', 'log2'*, and '*None*'. Finally, for the Decision Tree, in the criterion, we look at '*entropy'* and '*gini'*. We do this for Random Forest too, but we additionally look at n_estimators, which have *3, 7, 10,* and *15*. During testing, it became clear to have these values as minimal as possible so that the test time wouldn't take too long. Therefore, these were the optimal values selected to explore different levels of model complexity and performance. Accordingly, the max_depth parameter controls the maximum depth of the tree, and the deeper trees can capture more detailed patterns but risk overfitting. Therefore, it is crucial to choose a parameter that balances the model best; a range of depths was chosen to do exactly that. The parameters 5 and 10 allow for complexity without growing too deep, whilst the None option allows the tree to grow fully until it reaches a stop criterion or a leaf node.

Model Selection Results (Letter Recognition Dataset)

Decision Trees

From the Decision Tree model that was designed from scratch, the best parameters were estimated to be: *None* (best depth), *sqrt* (best feature), and *gini* (best criterion). The accuracy was approximately 89.6% with a runtime of 2 minutes and 32 seconds.

The *None* parameter means that there was no limit to how many nodes the model created, whilst the *sqrt* parameter means that we consider the square root of the total number of features. Having *gini* as the best criterion means that we get the best impurity measure using that method in our model.

For the test accuracy, it was estimated to be 93.5%.

Random Forest

From the Random Forest model that was implemented from scratch, the best parameters were estimated to be: *10* (best depth), *log2* (best feature), 15 (best estimators), and *entropy* (best criterion). The accuracy was approximately 94.2% with a runtime of 68 minutes and 33 seconds.

In this case, the depth was set to 10, meaning that there was a limit to how many nodes the model created, whilst the log2 parameter meant that it presented the total number of features by taking log2 of it. The best criterion was set to entropy in this case, which means that it was the best impurity measure in this model. Additionally, it was estimated that 15 was the best number estimator, which is notably the highest number of the given parameters.

For the test accuracy, it was estimated to be 94.0%.

Comparing Decision Trees and Random Forests

Comparing the two models, we get the results represented in Table 1:

Hyperparameters	Decision Tree	Random Forest
Max Depth	None	10
Criterion	"gini"	"entropy"
Max Feature	"sqrt"	"log2"
n-estimators	-	15
Validation Accuracy	89.6%	94.2%
Test Accuracy	93.5%	94.0%

Table 1: Comparing Decision Tree and Random Forest

Comparison to Existing Implementations

From the scikit-learn, we implement the existing models "DecisionTree" and "RandomForest" and use the models to compare with ours. It is the same scoring system and dataset that is used when the models are being run and compared. To showcase the different settings and accuracies, the results are summarized in Table 2.

	Ours	**Ours**	**SKlearn**	**SKlearn**
Parameters	**Decision Tree**	**Random Forest**	**Decision Tree**	**Random Forest**
Max Depth	None	10	None	30
Criterium	"gini"	"entropy"	"entropy"	"gini"
Max Features	"sqrt"	"log2"	"log2"	"log2"
N_estimators	-	15	-	15
Validation accuracy	89.6%	94.2%	91.2%	95.3%
Test accuracy	93.5%	94.0%	91.0%	95.4%

Table 2: Comparing the custom model with a built-in model

As one can see, the results from the implemented method from scratch and the ones from sklearn are very similar, with only a few percentage points in difference. Comparing the Decision Tree models, ours has a lower validation accuracy than Sklearn's, with a difference of 1.6% points. Whilst in the testing accuracy, our model had an increase to 93.5% whilst sklearn's decreased by 02% points.

This means that both the models give the same performance, but sklearn has a slightly better validation, most likely because its more optimized. Whilst my own implementation coincidentally had a more accurate hit on the testset. This means that my implementation works just as well, and has not overfitted more than the sklearn model.

Focusing on the Random Forest model, our model had 94.2% on the validation accuracy, and the sklearn had 95.3%. Both accuracies were separated by almost 1% point. Our model had a decrease of 0.2% points in the test accuracy, while Sklearn's had an increase of 0.1% points.

This shows that both the models are strong, but sklearn is slightly ahead when it comes to validation and test. As previously stated, this makes sense since sklearn is much more optimalised and can handle parameters faster and more precise than my own implementation.

Overall, the differences were very slim which, and the small differences can come from coincidences in train/test-splits, implimitation details such as how best split is found, or that sklearn uses very optimalized routines. In addition, the custom model is educational, but lacks some of the advanced features and optimizations found in sklearn. To be precise, sklearn optimizes numerical precisions and reduce overfitting using techniques such as pruning, which was not implemented in the custom model. In sklearn's Random Forest, bootstrap sampling is performed very efficiently, and it uses out-of-bag (OOB) evaluation. which helps greatly in the estimation of the generalization error more accurately. Whilst the custom one may not handle these processes as effectively, especially since it took over an hour to estimate. Additionally, sklearn is better equipped to manage missing values, noise, and data imbalances. [1]

On the other hand, the custom model performed quite well compared to an industrialized model that delivers high accuracy and F1 scores due to its sophisticated library. As previously stated, the differences in percentage points were quite minimal, and interestingly, the custom Decision Tree and Random Forest decreased in percentage points whilst sklearn's increased.

Feature Importance

In the custom permutation feature importance, the implementation started by first calculating the model's baseline accuracy on the test set. Then, for each feature, it repeatedly permuted or shuffled its values while keeping all other features unchanged. After each permutation, the accuracy of the model

was measured again. The decrease in accuracy compared to the baseline represents the importance of the given feature. The process was repeated 30 times per feature as it was given in the task. This was done to reduce variance. In addition, the mean and standard deviation of the importance scores were computed.

From the results in Figure 4, it is clear that features xy2br and yegc are by far the most important for the model, as they caused a 25-28% drop in accuracy. While features such as ybar and yegxr have a moderate influence, the others contribute little to the model's performance.

These findings indicate that the model relies heavily on a small subset of features to make accurate predictions. By creating this plot, it can guide feature selection and improve interpretability by showing which features have the biggest impact. However, permutation importance has limits. It can be computationally expensive since it requires many re-evaluations of the model, and it may underestimate the importance of correlated features.

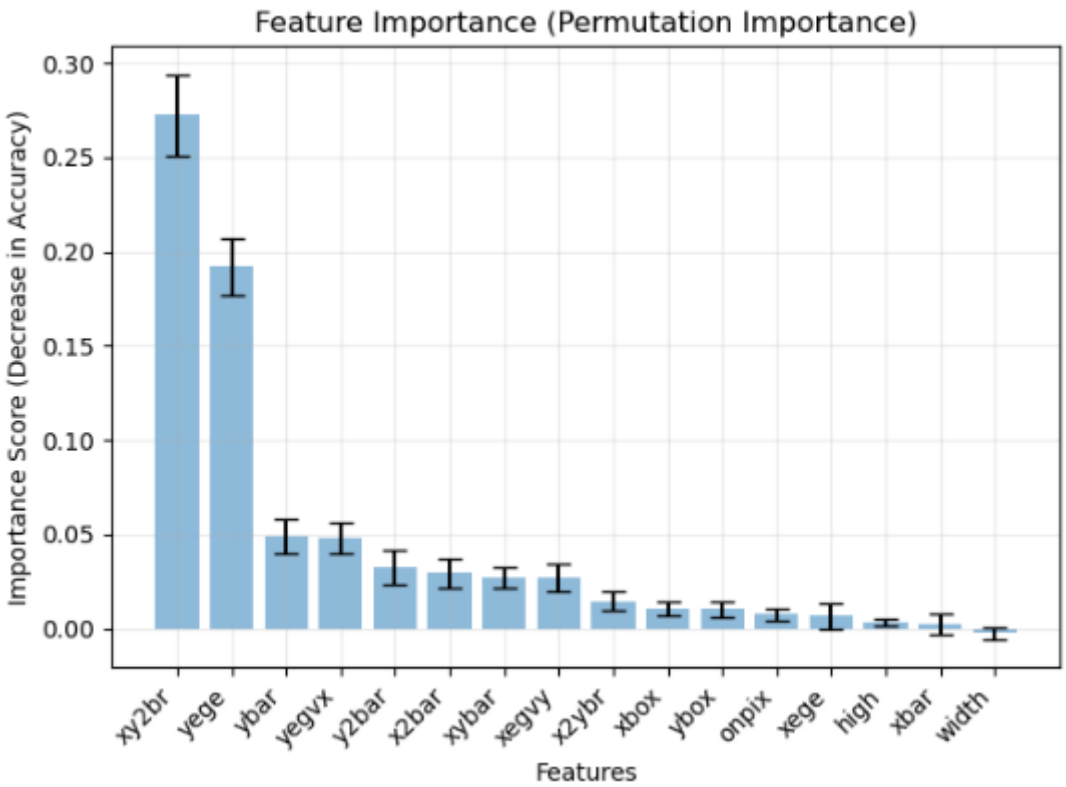

Figure 4: Boxplot over decrease in accuracy for the different features in "letters.csv"

Conclusion

In this project, decision trees and random forests were successfully implemented from scratch and applied to the Letter Recognition dataset. Through this process, a deeper understanding of how these

models function internally was gained, and how design choices such as impurity measures, maximum depth, and feature selection affect performance.

The results from our custom implementations compared to optimized implementations in scikit-learn show that they perform similarly, with only minor differences in accuracy. While scikit-learn achieves slightly higher validation and test performance due to its optimizations, the custom implementation demonstrates that the core algorithms can be built and tuned effectively from first principles. Permutation feature importance further revealed that only two features had a strong impact on the model predictions, which highlights both interpretability and the limitations of this method.

Looking back, there are a couple of things that would have been approached differently if done again. The biggest problem that occurred was during hyperparameter tuning for Random Forest on the custom model. If done again, there'd be more extensive hyperparameter tuning by testing a wider range of values. It was also learned that the trade-off between model complexity and how long it takes to run correlates. Whilst increasing the number of trees or n_estimators improved performance, it also made the training time much longer. Running the custom models took significantly more time than the built-in ones from scikit-learn. Building models from scratch definitely helps with understanding and has a more educational value, but it can be much slower and less optimized than using built-in implementations.

Overall, the project was quite educational as it taught a lot about developing machine learning models, tuning hyperparameters, impurity measures such as entropy and gini, and evaluating performance. It shows how a minimal change or parameter can have a grand impact on the final results.

References:

[1] https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html